

Lab Exercise 6 – Future and CompletableFuture

Objective

In this lab, students will learn how to:

- Execute asynchronous tasks using Future
 - Retrieve results from background tasks
 - Understand the limitation of Future
 - Execute asynchronous tasks using CompletableFuture
 - Chain asynchronous operations
 - Process results without manually waiting
-

Scenario

A university is building a **Student Result Processing System**.

Whenever a student's exam is completed, the application should:

- Calculate total marks
- Determine the student's grade
- Display the result
- Notify that the result has been published

Since result calculation takes time, it should execute asynchronously.

Business Requirements

The application should:

- Calculate marks in the background.
 - Allow the main thread to continue working.
 - Retrieve the calculated marks.
 - Calculate the grade.
 - Display the final result.
 - Print a completion message.
-

Project Structure

FutureCompletableFutureLab

|

└─ src

|

— FutureDemo.java

— CompletableFutureDemo.java

Only **two Java classes** are required.

Class 1 – FutureDemo.java

Purpose

Demonstrates:

- ExecutorService
 - Callable
 - Future
 - Future.get()
 - isDone()
-

Complete Code

FutureDemo.java

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class FutureDemo {

    public static void main(String[] args) throws Exception {

        System.out.println("===== Future Example =====");

        // Create a thread pool with one thread
        ExecutorService executor = Executors.newSingleThreadExecutor();

        // Task to calculate marks
        Callable<Integer> calculateMarks = () -> {
```

```
        System.out.println("Calculating student marks...");

        Thread.sleep(4000);

        return 450;
    };

    // Submit task
    Future<Integer> future = executor.submit(calculateMarks);

    // Main thread continues working
    while (!future.isDone()) {

        System.out.println("Teacher is taking attendance...");

        Thread.sleep(1000);
    }

    // Get result
    Integer marks = future.get();

    System.out.println("\nStudent Marks : " + marks);

    executor.shutdown();
}
}
```

Expected Output

===== Future Example =====

Calculating student marks...

Teacher is taking attendance...

Teacher is taking attendance...

Teacher is taking attendance...

Teacher is taking attendance...

Student Marks : 450

Explanation of Main Code Blocks

1. Creating ExecutorService

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();
```

Creates one worker thread.

The worker thread executes background tasks.

2. Creating Callable

```
Callable<Integer> calculateMarks = () -> {  
    Thread.sleep(4000);  
    return 450;  
};
```

Callable represents a task that

- runs in another thread
 - returns a value
-

3. Submit Task

```
Future<Integer> future = executor.submit(calculateMarks);
```

The task starts executing immediately.

Java returns a Future object.

4. Check Completion

```
future.isDone()
```

Returns

false

while the task is running.

Returns

true

when completed.

5. Get Result

```
future.get();
```

Returns

450

If the task is not finished,

get() waits until it completes.

6. Shutdown

```
executor.shutdown();
```

Stops accepting new tasks and cleans up the thread pool.

Class 2 – CompletableFutureDemo.java

Purpose

Demonstrates:

- supplyAsync()
 - thenApply()
 - thenAccept()
 - thenRun()
-

Complete Code

CompletableFutureDemo.java

```
import java.util.concurrent.CompletableFuture;

public class CompletableFutureDemo {

    public static void main(String[] args) {

        System.out.println("===== CompletableFuture Example =====");

        CompletableFuture

            // Step 1
            .supplyAsync() -> {

                System.out.println("Calculating student marks...");

                try {
                    Thread.sleep(3000);
                }
                catch (Exception e) {
```



```
    }

    return 450;
})

// Step 2
.thenApply(marks -> {

    System.out.println("Calculating Grade...");

    if (marks >= 400)

        return "Grade A";

    else

        return "Grade B";
})

// Step 3
.thenAccept(grade -> {

    System.out.println("Student Grade : " + grade);

})

// Step 4
.thenRun() -> {

    System.out.println("Result Published Successfully.");

})

.join();
}
```

Expected Output

===== CompletableFuture Example =====

Calculating student marks...

Calculating Grade...

Student Grade : Grade A

Result Published Successfully.

Explanation of Main Code Blocks

1. `supplyAsync()`

```
CompletableFuture.supplyAsync(() -> {  
  
    return 450;  
  
});
```

Starts a background task.

Returns

450

after completion.

2. thenApply()

```
.thenApply(marks -> {  
  
    return "Grade A";  
  
})
```

Receives

450

Transforms it into

Grade A

Returns another `CompletableFuture`.

3. thenAccept()

```
.thenAccept(grade -> {  
  
    System.out.println(grade);  
  
})
```

Uses the result.

Does not return another value.

Used for

- printing

- saving
 - logging
-

4. **thenRun()**

```
.thenRun() -> {  
  
    System.out.println("Result Published");  
  
});
```

Runs another task after completion.

It

- receives no input
 - returns no value
-

5. **join()**

```
.join();
```

Waits until the entire pipeline finishes.

Without `join()`, the application may terminate before the asynchronous tasks complete.